CS202: Software Tools and Techniques for CSE

Lecture 4

Shouvick Mondal

shouvick.mondal@iitgn.ac.in

January 2025

Outline

- Introduction
- Complexity measures
  - Using algebraic expressions
  - Using basic paths
- CFGs for multiple functions
- Simplification of the complexity calculation

Introduction

- Issues with software systems
  - Testability
  - Maintainability
- Time and Cost
  - Time: 50% testing
  - Cost: 50% maintenance
- We expect a mathematical technique that would
  - Provide a quantitative basis for modularization of programs
  - Identify software modules that are difficult to test or maintain

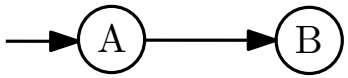
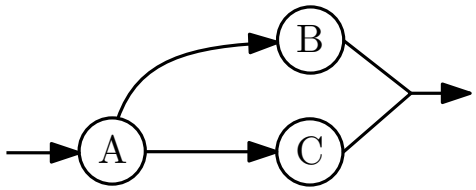
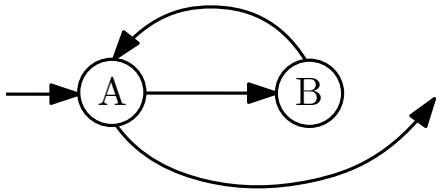
Introduction

- What we do not expect
  - Modularization by specifying physical size of code
  - Extreme example:- 1 Module = 50 lines, code contains 25 IF..THEN..IF..THEN..
  - Drawback - Physically small but untestable
- What we should look for
  - Decision structure of the associated CFG
  - What structured programming is
  - What structured programming is not
  - Design for testability
  - Design for maintainability

Complexity measures

- Measure and control number of paths through a program
  - Problem: back edges $\rightarrow \infty$ paths
- Two methods
  - Method 1: using algebraic expressions
  - Method 2: using basic paths - focus of this talk

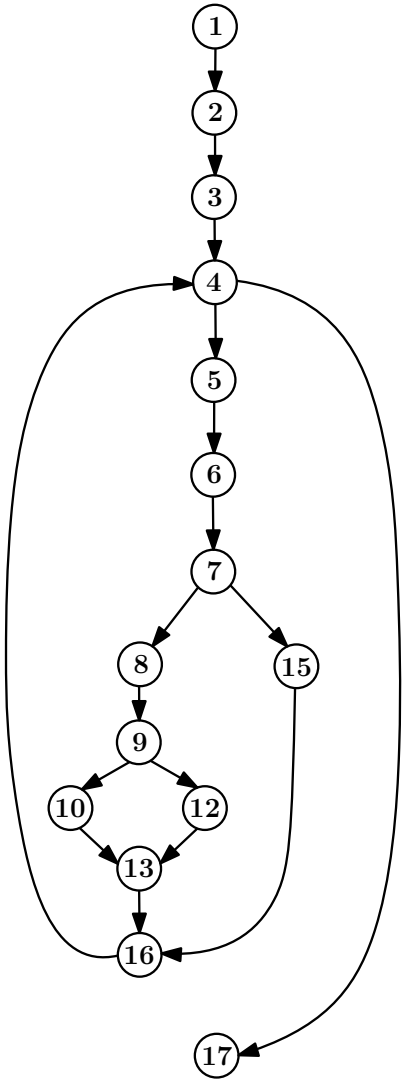
Method 1: using algebraic expressions

CONSTRUCT	CONTROL FLOW	C (CONSTRUCT)
SEQUENCE A;B		$C(A) \times C(B)$
IF A THEN B ELSE C		$C(A) \times [C(B) + C(C)]$
WHILE A DO B		$C(A) + [C(A) \times C(B)]^{\alpha}\ C(A)$

α is the number of iterations in a loop

Method 1: using algebraic expressions

```
1   void search(int *a,int n,int item)
2   {
3       int f=0,l=0,h=n,mid;
4       while(h>=l && f==0)
5       {
6           mid=(h+l)/2;
7           if(a[mid]!=item)
8           {
9               if(a[mid]<item)
10                  l=mid+1;
11              else
12                  h=mid-1;
13          }
14          else
15              f=1;
16      }
17  }
```



C(search)=$\{1 + (3)^{\alpha}\}$

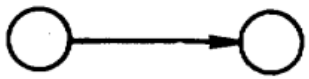
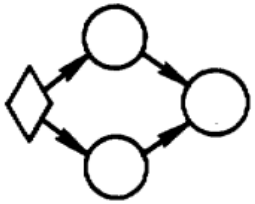
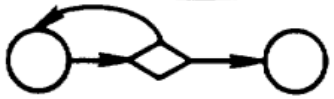
Method 2: using basic paths

- We only need to look for the set of linearly independent paths, B, in the program
- This maximal set B is called the Basis
- $|B|$ is the minimum number of paths that must be tested
- The set B in necessary and sufficient

Cyclomatic complexity/number, $V(G)$

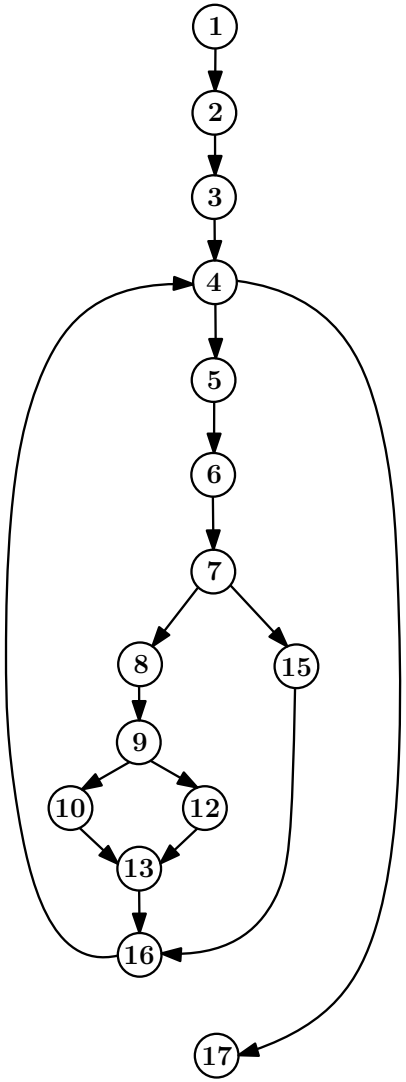
- $V(G)$ is calculated as $V(G) = e - n + 2 * p$
  - e is the # of edges in G
  - n is the # of vertices in G
  - p is the # of connected components in G
- $V(G)$ is the maximum # of linearly independent paths in G
- It is the size of the basis set i.e., $V(G) = |B|$
- $V(G)$ depends only on the decision structure of G

Basic control flow graphs and their complexities

CONTROL STRUCTURE	CYCLOMATIC COMPLEXITY *v = e - n + 2p
SEQUENCE 	v = 1 - 2 + 2 = 1
IF THEN ELSE 	v = 4 - 4 + 2 = 2
WHILE 	v = 3 - 3 + 2 = 2
UNTIL 	v = 3 - 3 + 2 = 2

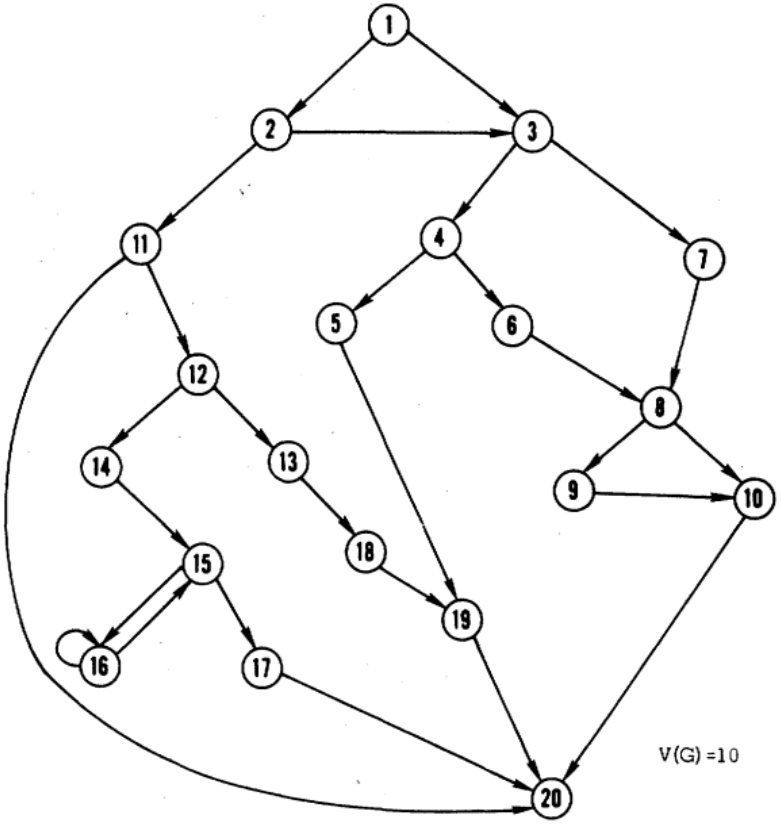
Re-visiting void search(int*, int, int)

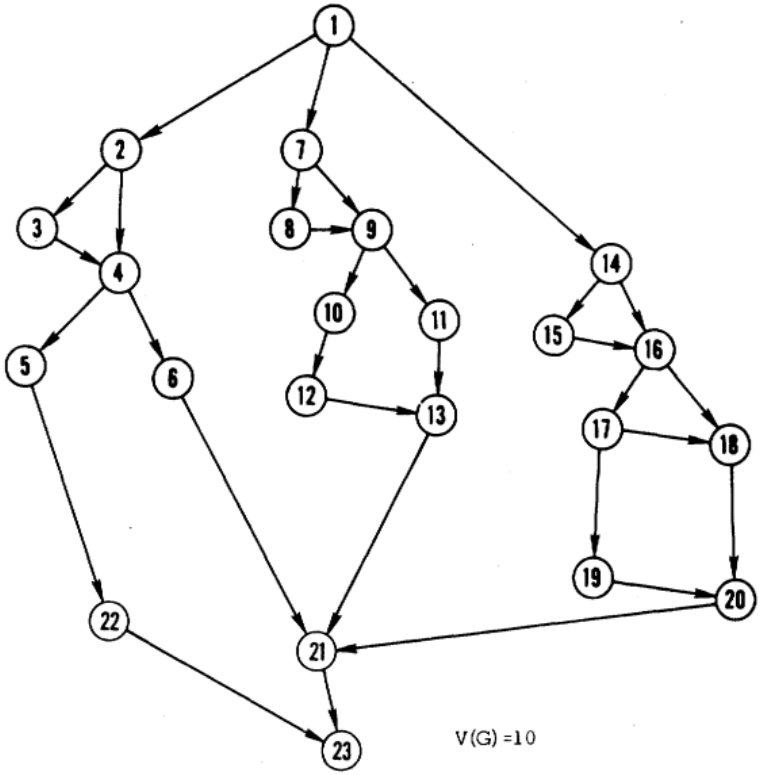
```
1   void search(int *a,int n,int item)
2   {
3       int f=0,l=0,h=n,mid;
4       while(h>=l && f==0)
5       {
6           mid=(h+l)/2;
7           if(a[mid]!=item)
8           {
9               if(a[mid]<item)
10                  l=mid+1;
11              else
12                  h=mid-1;
13          }
14          else
15              f=1;
16      }
17  }
```



$$V(G_{search}) = e - n + 2 = 17 - 15 + 2 = 4$$

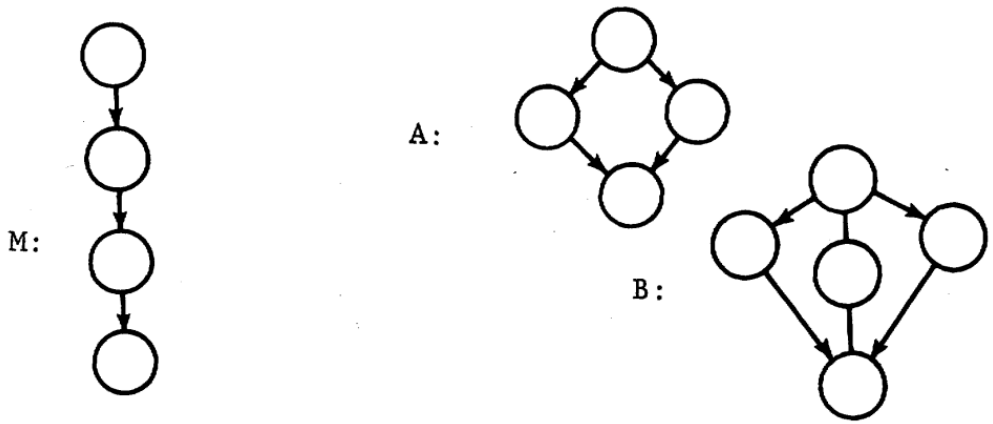
Some CFGs of $V(G) = 10$





CFGs for multiple functions

- What we have considered so far
  - All vertices reachable from ENTRY
  - EXIT is reachable from all vertices
  - Single connected component, $p = 1$
- For CFG with multiple functions (components)

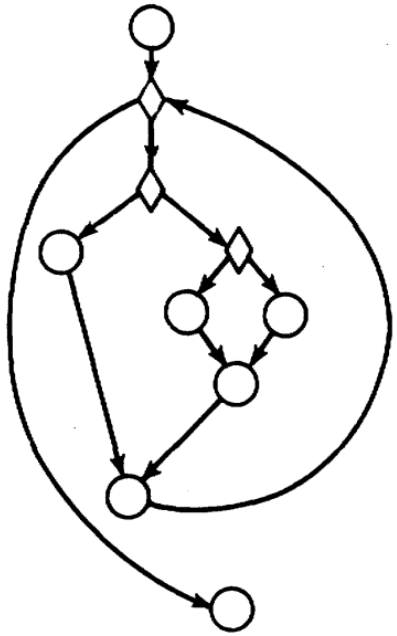


  - $V(M \cup A \cup B) = e - n + 2 * p = 13 - 13 + 2 * 3 = 6$
  - $V(M) + V(A) + V(B) = 1 + 2 + 3 = 6$

Simplification 1: single component graphs

- The calculation $V(G) = e - n + 2 * p$ can be quite tedious for large CFGs
- If π is the # of predicate nodes, then $V(G) = \pi + 1$

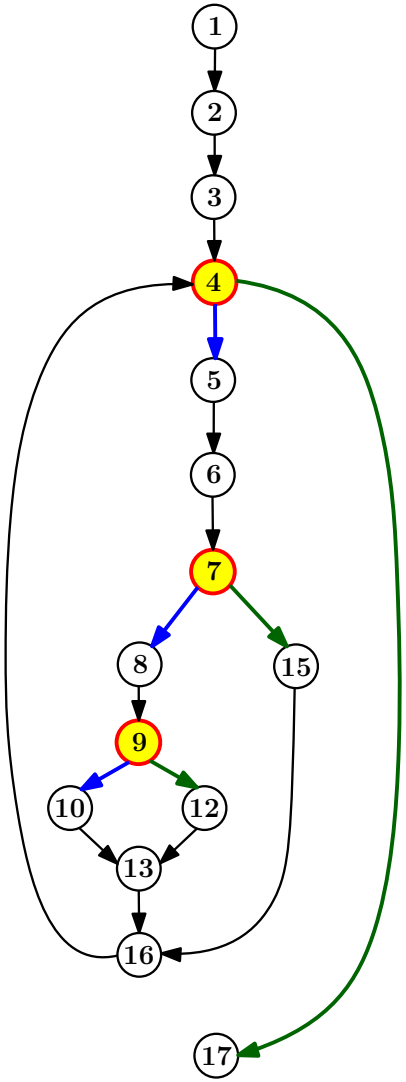
G:



- $V(G) = \pi + 1 = 3 + 1 = 4$

Re-visiting void search(int*, int, int)

```
1   void search(int *a,int n,int item)
2   {
3       int f=0,l=0,h=n,mid;
4       while(h>=l && f==0)
5       {
6           mid=(h+l)/2;
7           if(a[mid]!=item)
8           {
9               if(a[mid]<item)
10                  l=mid+1;
11              else
12                  h=mid-1;
13          }
14          else
15              f=1;
16      }
17  }
```



$$V(G_{search}) = \pi + 1 = 3 + 1 = 4$$

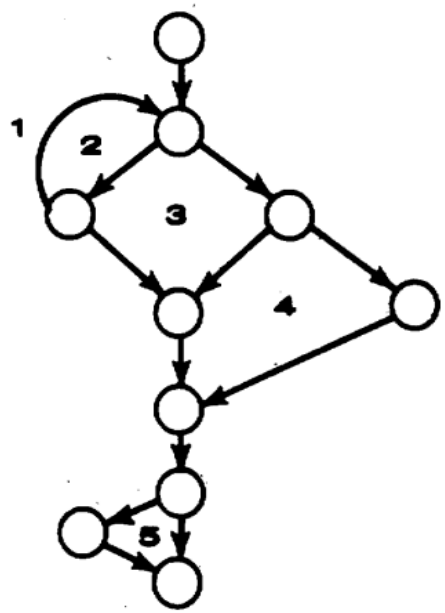
Simplification 1: single component graphs

- {IF “C1 AND C2” THEN}→{IF C1 THEN IF C2 THEN}. Note that the later has two predicates
- For the CASE construct with N cases
  - Use N-1 for the # of conditions
  - The simulation with IFs will have N-1 predicates
- Therefore, for the purposes of testing # of conditions is more relevant than # of predicates

Simplification 2: single component graphs

- Euler's formula
  - If G is a connected planar graph, then, $n - e + r = 2$
  - Re-ordering the terms, we get, $r = e - n + 2$
  - # of regions in $G = V(G)$

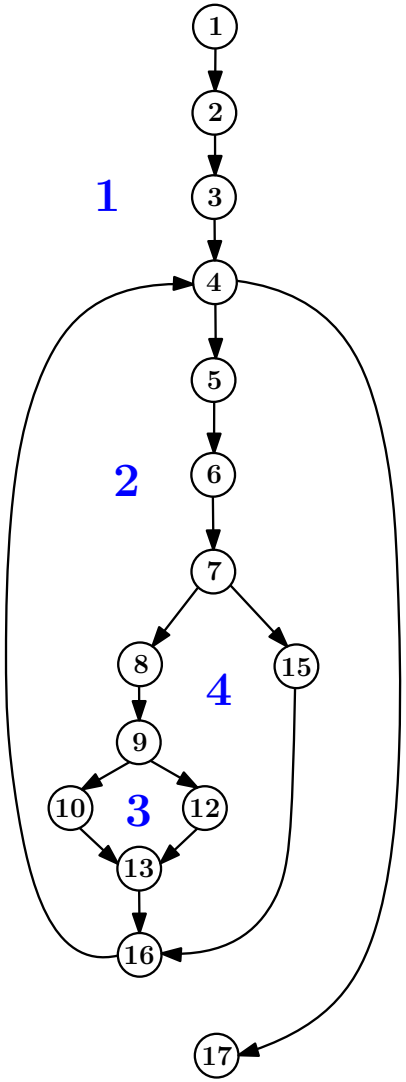
G:



v(G) = 5

Re-visiting void search(int*, int, int)

```
1   void search(int *a,int n,int item)
2   {
3       int f=0,l=0,h=n,mid;
4       while(h>=l && f==0)
5       {
6           mid=(h+l)/2;
7           if(a[mid]!=item)
8           {
9               if(a[mid]<item)
10                  l=mid+1;
11              else
12                  h=mid-1;
13          }
14          else
15              f=1;
16      }
17  }
```



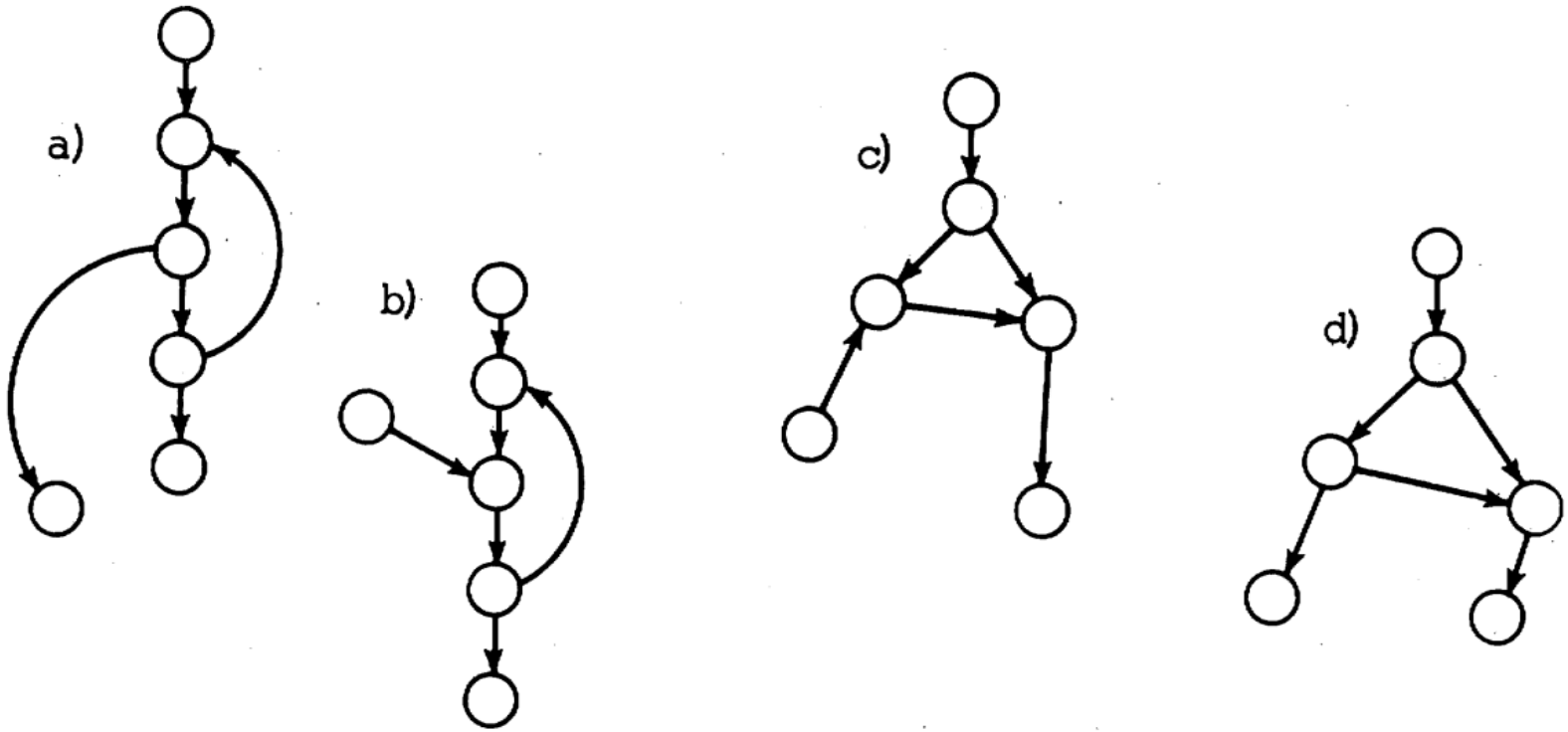
$$V(G_{search}) = 4$$

Non-structured programming

- What structured programming is and is not
  - Make programmers aware of a few syntactic constructs
  - Tell them to use only these constructs for structured programming
  - Drawback: we do not tell them what not to use i.e., what structured programming is not

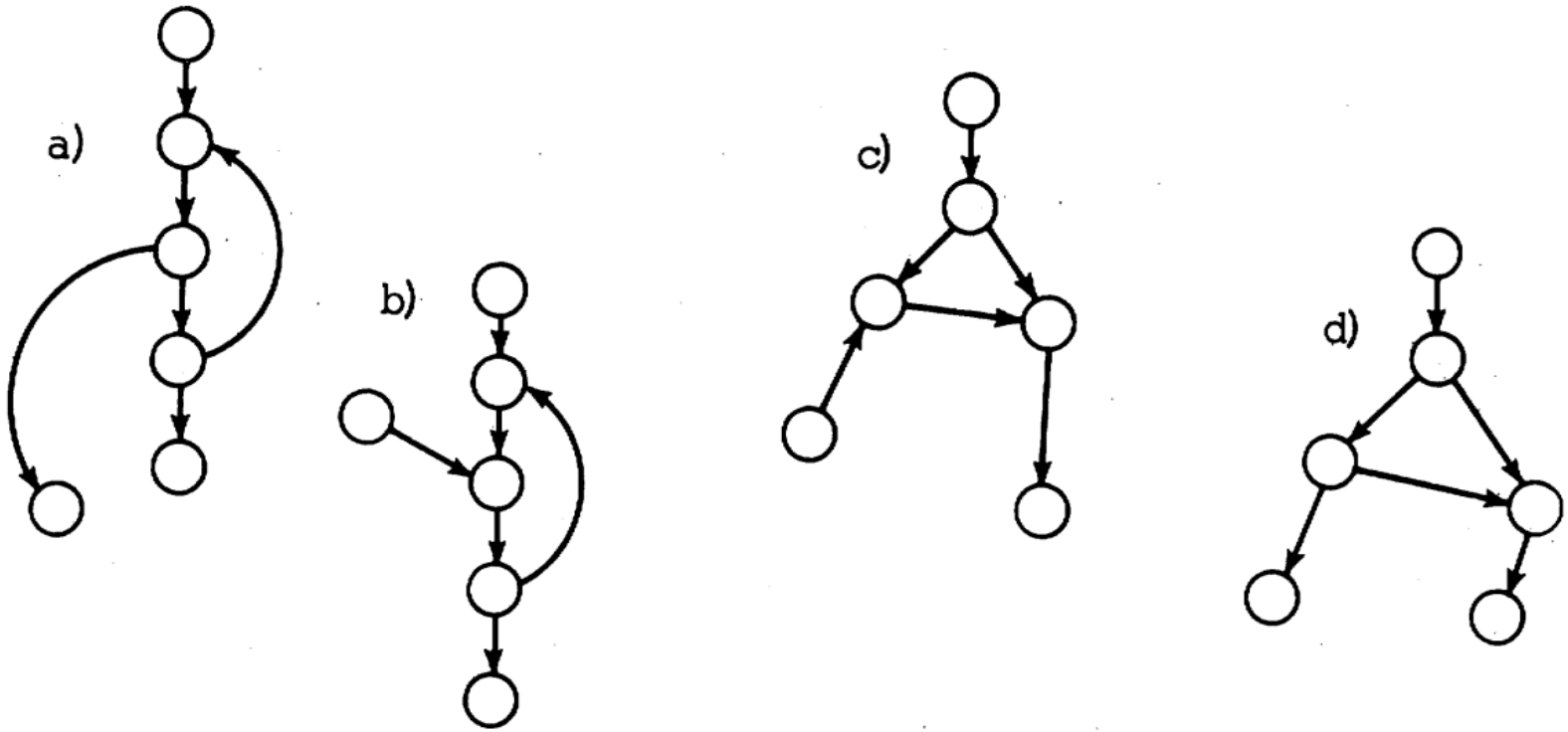
Non-structured programming

- A necessary and sufficient condition for a program to be non-structured is to have either of the following as a subgraph of its GFG



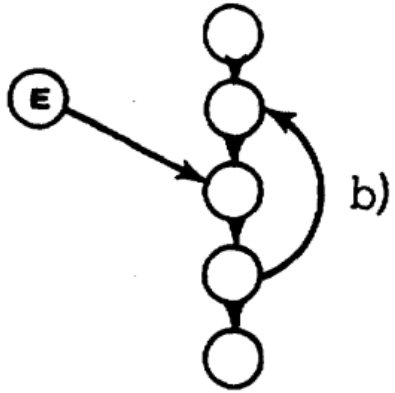
Non-structured programming

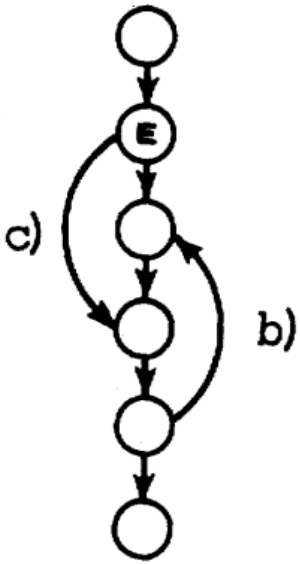
- A non-structured program cannot be just a little non-structured. That is any non-structured program must contain at least 2 of the graphs a)–d)

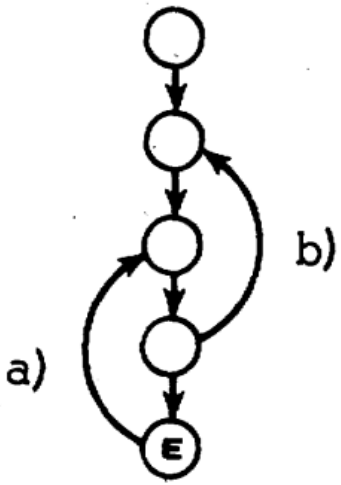


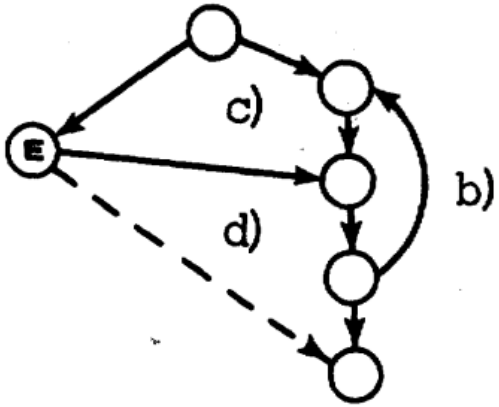
Non-structured programming

- Proof that b) cannot occur alone



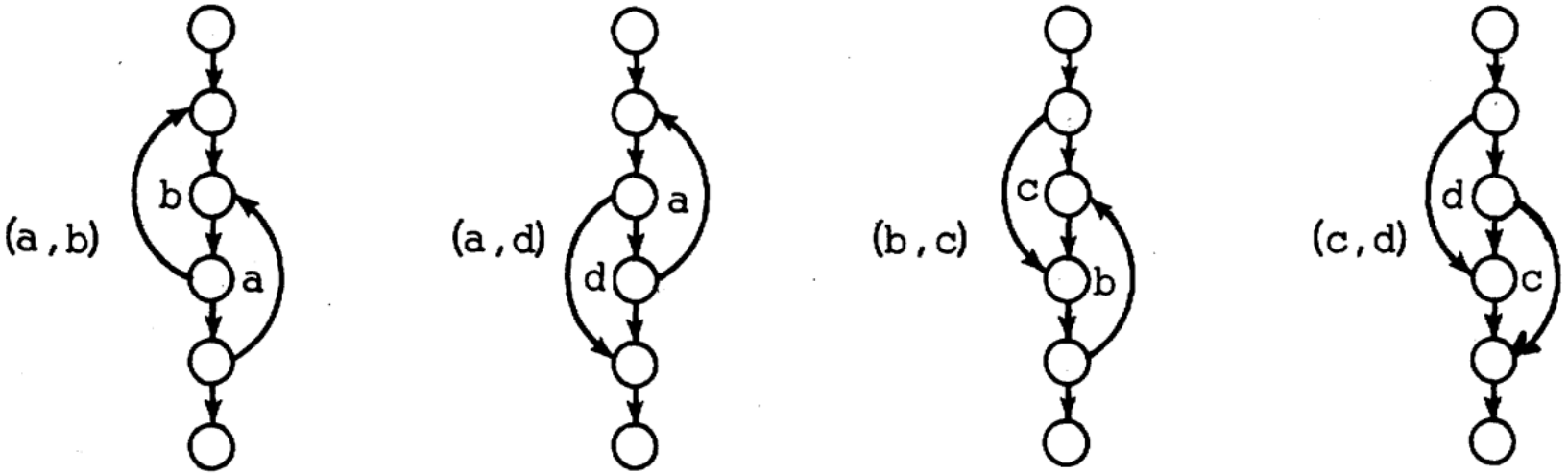






Non-structured programming

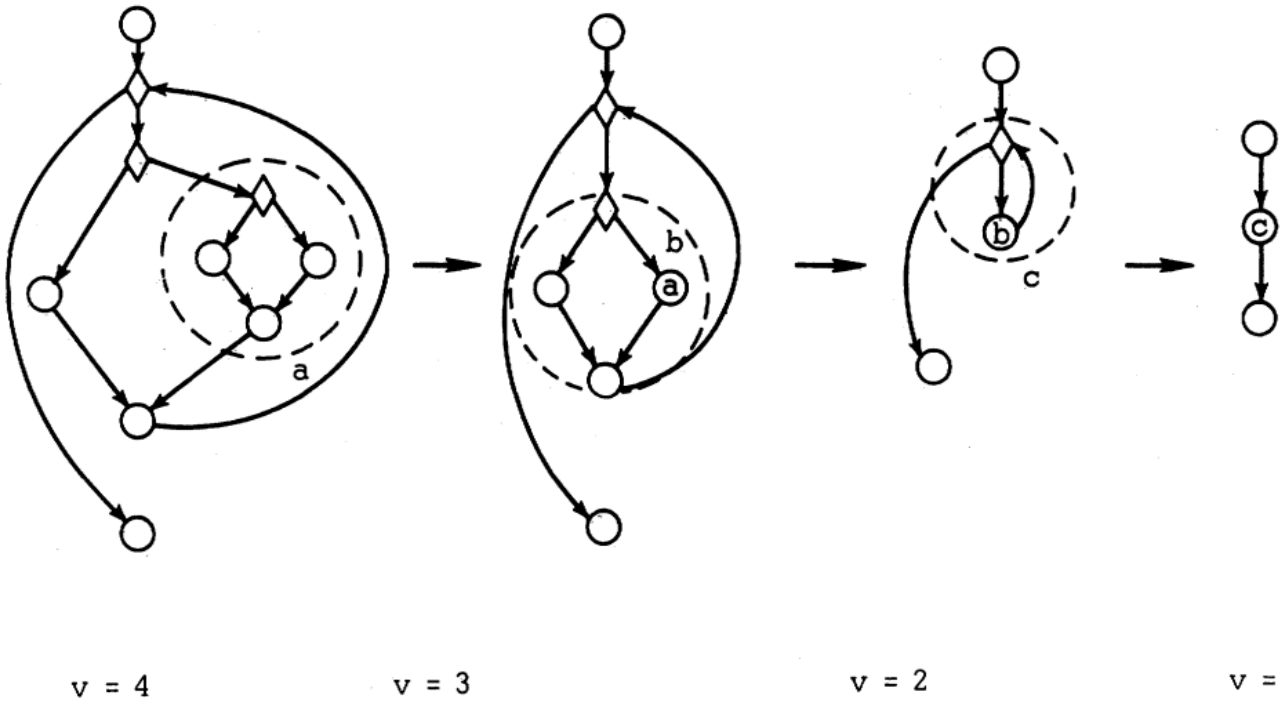
- The four basic types



- A necessary and sufficient condition for a program to be non-structured is that it contains at least one of: {(a,b), (a,d), (b,c), (c,d)}
- Cyclomatic complexity of a non-structured program is $\geqslant 3$

Structuredness in a program

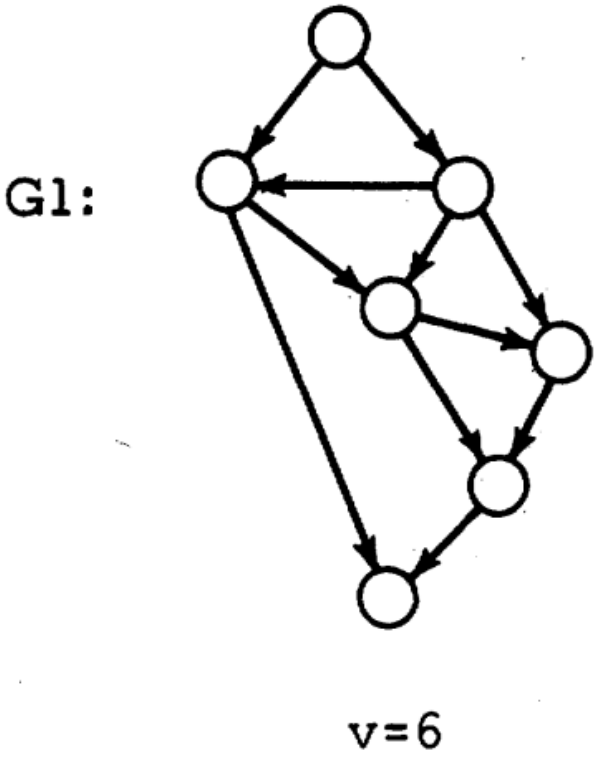
- Difficulty with non-structured graphs
  - There is no way they can be broken down into subgraphs with single entry and single exit
- A structured program is reducible to a program of unit complexity

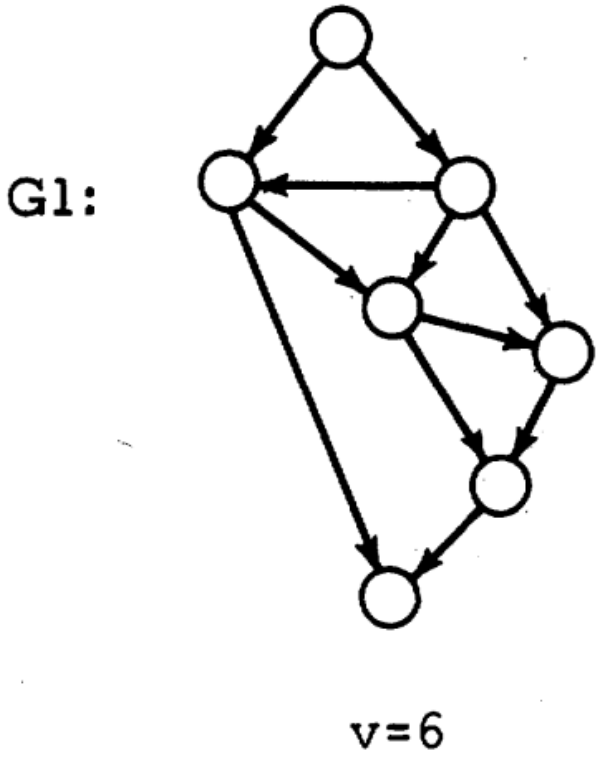


Structuredness in a program

- A measure of the lack of structure: essential complexity $eV(G) = V(G) - m$, where m is the # of proper subgraphs with unique entry and exit nodes
- This indicates that complexity can be reduced by m
- m can be thought of as a tuning parameter which helps in moving the graph (program) on the scale of structuredness

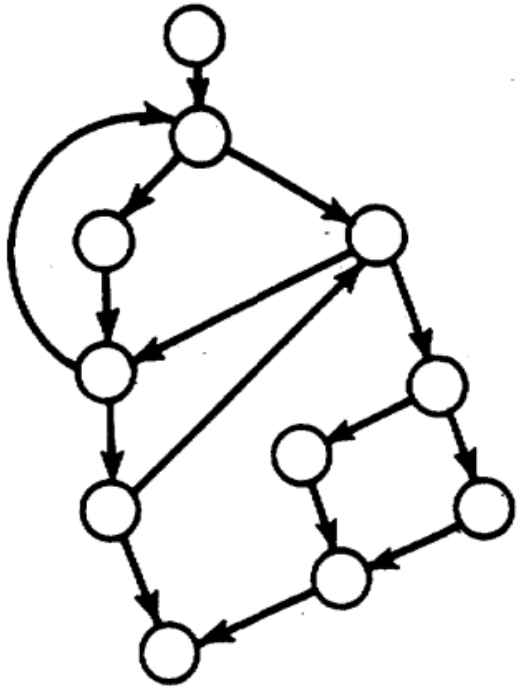
Structuredness in a program: $m = 0$





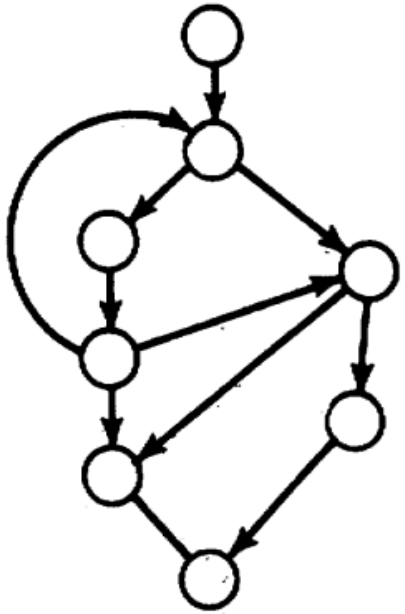
Structuredness in a program: $m = 1$

G2:



v=6

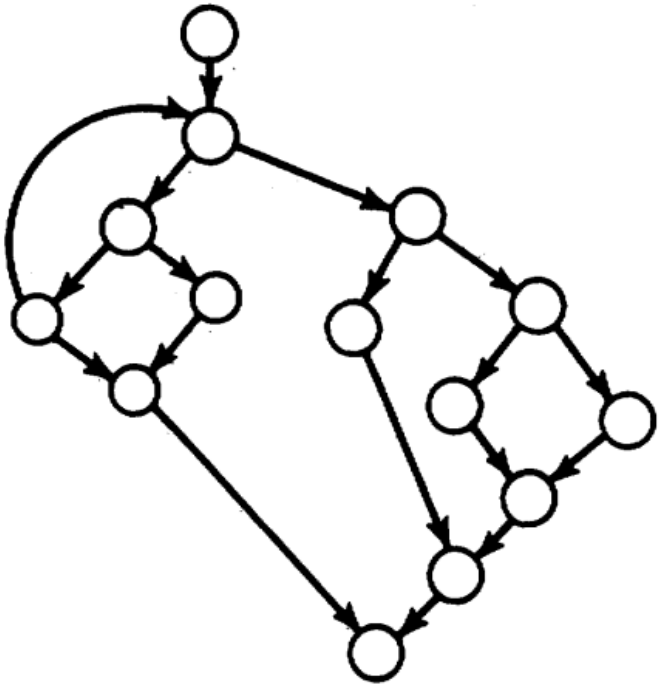
G2':



v(G2')=5

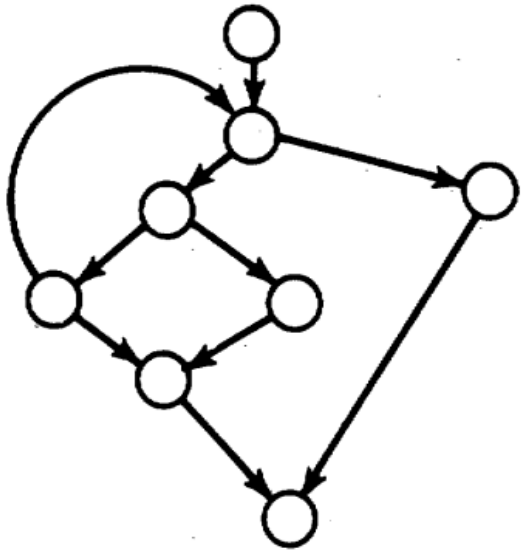
Structuredness in a program: $m = 2$

G3:



v=6

G3':



v(G3')=4

Conclusions

- While writing a program
  - Set a limit to the complexity of the corresponding CFG
  - If complexity exceeds, re-order control flow but preserve semantics
  - Decompose the CFG as far as possible: modularization
- Design for testability (once the program is written)
  - Measure the structuredness
  - Check for reducibility
- To test a program
  - Identify only the set linearly independent paths, B
  - B is maximal in the sense that it contains the maximum # of L.I.Ps
  - # of test-cases should be $\geqslant |B|$

References

- Thomas J. McCabe: *A Complexity Measure*. IEEE Transactions on Software Engineering, Vol. SE-2, No. 4, pp. 308–320, December 1976.